

Ultimate Tic Tac Toe — Source Code

PDF created from the code supplied in the chat. The supplied code ends inside the **endGame()** function, so the PDF preserves the provided portion and marks the missing remainder.

```
<!DOCTYPE html>
<html lang="en" class="dark">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ultimate Tic Tac Toe</title>

  <!-- Tailwind CSS CDN -->
  <script src="https://cdn.tailwindcss.com"></script>
  <script>
    tailwind.config = {
      darkMode: 'class',
      theme: {
        extend: {
          colors: {
            brand: {
              50: '#eef2ff',
              100: '#e0e7ff',
              500: '#6366f1',
              600: '#4f46e5',
              700: '#4338ca',
            },
            playerX: '#3b82f6',
            playerO: '#ec4899',
          },
          fontFamily: {
            sans: ['Outfit', 'sans-serif'],
          },
          animation: {
            'pop-in': 'popIn 0.25s cubic-bezier(0.175, 0.885, 0.32, 1.275) forwards',
            'pulse-glow': 'pulseGlow 2s infinite',
          },
          keyframes: {
            popIn: {
              '0%': { transform: 'scale(0.5)', opacity: '0' },
              '100%': { transform: 'scale(1)', opacity: '1' },
            },
            pulseGlow: {
              '0%, 100%': { opacity: '1', filter: 'drop-shadow(0 0 12px rgba(99, 102, 241, 0.6))' },
              '50%': { opacity: '0.7', filter: 'drop-shadow(0 0 4px rgba(99, 102, 241, 0.2))' },
            },
          },
        },
      },
    }
  </script>

  <!-- Google Fonts: Outfit -->
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Outfit:wght@400;500;600;700;800&display=swap" rel="stylesheet">

  <!-- Lucide Icons CDN -->
  <script src="https://unpkg.com/lucide@latest"></script>

  <!-- Canvas Confetti CDN -->
  <script src="https://cdn.jsdelivr.net/npm/canvas-confetti@1.9.2/dist/confetti.browser.min.js"></script>

  <style>
    body {
      font-family: 'Outfit', sans-serif;
      user-select: none;
    }

    /* Winning Line SVG stroke styling */
    .win-line {
      stroke-dasharray: 1000;
      stroke-dashoffset: 1000;
      animation: drawLine 0.5s ease-out forwards;
    }

    @keyframes drawLine {
      to {
        stroke-dashoffset: 0;
      }
    }

    /* Glassmorphism utility */
    .glass-card {
      background: rgba(255, 255, 255, 0.05);
      backdrop-filter: blur(16px);
      -webkit-backdrop-filter: blur(16px);
      border: 1px solid rgba(255, 255, 255, 0.1);
    }

    .light .glass-card {
      background: rgba(255, 255, 255, 0.7);
      border: 1px solid rgba(0, 0, 0, 0.08);
    }
  </style>
</head>

<body class="bg-slate-950 text-slate-100 dark:bg-slate-950 dark:text-slate-100 transition-colors duration-300 min-h-screen flex flex-col justify-between">
  <div class="w-full max-w-4xl mx-auto px-4 py-6">
    <!-- Header -->
    <header class="flex items-center justify-between mb-6">
      <div class="flex items-center gap-3">
```

```

<div class="p-2.5 bg-indigo-600/20 rounded-2xl border border-indigo-500/30 text-indigo-400">
  <i data-lucide="grid" class="w-7 h-7"></i>
</div>
<div>
  <h1 class="text-2xl font-extrabold tracking-tight bg-gradient-to-r from-indigo-400 via-purple-400 to-pink-400 bg-clip-text text-tra
nsparent">
    TIC TAC TOE
  </h1>
  <p class="text-xs text-slate-400 font-medium">Pro Edition</p>
</div>
</div>
<!-- Action Buttons -->
<div class="flex items-center gap-2">
  <!-- Sound Toggle -->
  <button id="sound-btn" class="p-2.5 rounded-xl glass-card hover:bg-slate-800/60 transition text-slate-300 hover:text-white" title="To
ogle Audio">
    <i id="sound-icon" data-lucide="volume-2" class="w-5 h-5"></i>
  </button>
  <!-- Theme Toggle -->
  <button id="theme-btn" class="p-2.5 rounded-xl glass-card hover:bg-slate-800/60 transition text-slate-300 hover:text-white" title="To
ogle Theme">
    <i id="theme-icon" data-lucide="sun" class="w-5 h-5"></i>
  </button>
  <!-- GitHub Link -->
  <a href="https://github.com" target="_blank" rel="noopener noreferrer" class="p-2.5 rounded-xl glass-card hover:bg-slate-800/60 trans
ition text-slate-300 hover:text-white" title="GitHub Repository">
    <i data-lucide="github" class="w-5 h-5"></i>
  </a>
</div>
</header>

<main class="grid grid-cols-1 md:grid-cols-12 gap-6 items-start">
  <!-- Left Column: Controls & Game Settings -->
  <section class="md:col-span-4 space-y-5">
    <!-- Game Mode Selection -->
    <div class="p-5 rounded-2xl glass-card space-y-4">
      <h2 class="text-sm font-semibold uppercase tracking-wider text-slate-400 flex items-center gap-2">
        <i data-lucide="gamepad-2" class="w-4 h-4 text-indigo-400"></i> Game Mode
      </h2>
      <div class="grid grid-cols-2 gap-2 p-1 bg-slate-900/60 rounded-xl border border-slate-800">
        <button id="mode-pvp" class="py-2.5 px-3 rounded-lg font-medium text-xs transition flex items-center justify-center gap-1.5 bg-in
digo-600 text-white shadow-md">
          <i data-lucide="users" class="w-4 h-4"></i> 2 Players
        </button>
        <button id="mode-ai" class="py-2.5 px-3 rounded-lg font-medium text-xs transition flex items-center justify-center gap-1.5 text-s
late-400 hover:text-white">
          <i data-lucide="bot" class="w-4 h-4"></i> vs AI
        </button>
      </div>
      <!-- Difficulty Selector (hidden in PVP) -->
      <div id="ai-difficulty-container" class="space-y-2 hidden">
        <label class="text-xs font-medium text-slate-400">AI Difficulty</label>
        <select id="ai-difficulty" class="w-full bg-slate-900 border border-slate-800 text-slate-200 text-xs rounded-xl p-2.5 focus:outli
ne-none focus:border-indigo-500 transition">
          <option value="easy">Casual (Easy)</option>
          <option value="medium">Challenging (Medium)</option>
          <option value="unbeatable" selected>Unbeatable AI (Minimax)</option>
        </select>
      </div>
      <!-- Symbol Theme Selector -->
      <div class="space-y-2">
        <label class="text-xs font-medium text-slate-400">Marker Theme</label>
        <select id="marker-theme" class="w-full bg-slate-900 border border-slate-800 text-slate-200 text-xs rounded-xl p-2.5 focus:outlin
e-none focus:border-indigo-500 transition">
          <option value="standard">Standard (X / O)</option>
          <option value="cyber">Cyberpunk (■ / ■)</option>
          <option value="space">Space (■ / ■)</option>
          <option value="royale">Royale (■ / ■)</option>
        </select>
      </div>
    </div>
    <!-- Scoreboard Card -->
    <div class="p-5 rounded-2xl glass-card space-y-4">
      <div class="flex items-center justify-between">
        <h2 class="text-sm font-semibold uppercase tracking-wider text-slate-400 flex items-center gap-2">
          <i data-lucide="trophy" class="w-4 h-4 text-amber-400"></i> Scoreboard
        </h2>
        <button id="reset-score-btn" class="text-xs text-slate-400 hover:text-rose-400 transition flex items-center gap-1">
          <i data-lucide="rotate-ccw" class="w-3 h-3"></i> Reset
        </button>
      </div>
      <div class="grid grid-cols-3 gap-3 text-center">
        <div class="p-3 bg-slate-900/50 rounded-xl border border-indigo-500/20">
          <div id="score-p1-label" class="text-xs font-medium text-blue-400 truncate">Player X</div>
          <div id="score-p1" class="text-2xl font-extrabold text-blue-400 mt-1">0</div>
        </div>
        <div class="p-3 bg-slate-900/50 rounded-xl border border-slate-800">
          <div class="text-xs font-medium text-slate-400">Ties</div>
          <div id="score-ties" class="text-2xl font-extrabold text-slate-300 mt-1">0</div>
        </div>
        <div class="p-3 bg-slate-900/50 rounded-xl border border-pink-500/20">
          <div id="score-p2-label" class="text-xs font-medium text-pink-400 truncate">Player O</div>
          <div id="score-p2" class="text-2xl font-extrabold text-pink-400 mt-1">0</div>
        </div>
      </div>
    </div>
  </section>

```

```

<!-- Right Column: Main Game Canvas -->
<section class="md:col-span-8 flex flex-col items-center justify-center space-y-6">
  <div id="status-banner" class="w-full p-4 rounded-2xl glass-card flex items-center justify-between border-l-4 border-blue-500 transition-all duration-300">
    <div class="flex items-center gap-3">
      <span id="turn-icon-container" class="text-2xl font-extrabold text-blue-400">X</span>
      <div>
        <p id="turn-text" class="text-sm font-semibold text-slate-200">Player X's Turn</p>
        <p id="turn-subtext" class="text-xs text-slate-400">Click a tile to place marker</p>
      </div>
    </div>
    <div id="timer-badge" class="hidden text-xs font-mono px-3 py-1 bg-slate-800 rounded-full border border-slate-700 text-slate-300">
      00:00
    </div>
  </div>
  <div class="relative w-full max-w-[380px] aspect-square p-3 bg-slate-900/80 rounded-3xl border border-slate-800 shadow-2xl">
    <svg id="winning-line-svg" class="absolute inset-0 w-full h-full pointer-events-none z-20 overflow-visible" xmlns="http://www.w3.org/2000/svg">
      <line id="win-line" class="win-line hidden" x1="0" y1="0" x2="0" y2="0" stroke="#f43f5e" stroke-width="8" stroke-linecap="round" />
    </svg>
    <div id="board" class="grid grid-cols-3 gap-3 w-full h-full">
      <!-- Tiles generated by JavaScript -->
    </div>
  </div>
  <div class="flex items-center gap-3">
    <button id="undo-btn" disabled class="px-4 py-2.5 rounded-xl glass-card text-xs font-medium text-slate-400 hover:text-white disabled:opacity-40 disabled:cursor-not-allowed transition flex items-center gap-2">
      <i data-lucide="undo-2" class="w-4 h-4"></i> Undo Move
    </button>
    <button id="restart-btn" class="px-5 py-2.5 rounded-xl bg-gradient-to-r from-indigo-600 to-purple-600 hover:from-indigo-500 hover:to-purple-500 text-white text-xs font-bold shadow-lg shadow-indigo-500/20 transition transform active:scale-95 flex items-center gap-2">
      <i data-lucide="rotate-cw" class="w-4 h-4"></i> New Match
    </button>
  </div>
</section>
</main>
<footer class="mt-12 py-6 border-t border-slate-800/60 text-center text-xs text-slate-500 flex flex-col sm:flex-row items-center justify-between gap-3">
  <p>Built for GitHub Pages • Single File HTML5 Game</p>
  <p class="flex items-center gap-1">
    <span>Made with</span> <i data-lucide="heart" class="w-3.5 h-3.5 text-rose-500 fill-rose-500"></i> <span>for Open Source</span>
  </p>
</footer>
</div>
<script>
class SoundSynth {
  constructor() {
    this.ctx = null;
    this.enabled = true;
  }
  init() {
    if (!this.ctx) {
      this.ctx = new (window.AudioContext || window.webkitAudioContext)();
    }
  }
  playTone(freq, type = 'sine', duration = 0.1, gainVal = 0.15) {
    if (!this.enabled) return;
    try {
      this.init();
      const osc = this.ctx.createOscillator();
      const gain = this.ctx.createGain();
      osc.type = type;
      osc.frequency.setValueAtTime(freq, this.ctx.currentTime);
      gain.gain.setValueAtTime(gainVal, this.ctx.currentTime);
      gain.gain.exponentialRampToValueAtTime(0.0001, this.ctx.currentTime + duration);
      osc.connect(gain);
      gain.connect(this.ctx.destination);
      osc.start();
      osc.stop(this.ctx.currentTime + duration);
    } catch (e) {
      console.warn("Audio play blocked", e);
    }
  }
  move() {
    this.playTone(440, 'sine', 0.08, 0.1);
  }
  win() {
    if (!this.enabled) return;
    const notes = [523.25, 659.25, 783.99, 1046.50];
    notes.forEach((freq, idx) => {
      setTimeout(() => this.playTone(freq, 'triangle', 0.25, 0.2), idx * 100);
    });
  }
  draw() {
    if (!this.enabled) return;
    this.playTone(300, 'sawtooth', 0.15, 0.1);
    setTimeout(() => this.playTone(250, 'sawtooth', 0.2, 0.1), 120);
  }
}
const audio = new SoundSynth();
const SYMBOL_SETS = {
  standard: { x: 'X', o: 'O' },
  cyber: { x: '■', o: '■' },
  space: { x: '■', o: '■' },

```

```

royale: { x: '■', o: '■' }
};

const WINNING_COMBOS = [
  [0, 1, 2], [3, 4, 5], [6, 7, 8],
  [0, 3, 6], [1, 4, 7], [2, 5, 8],
  [0, 4, 8], [2, 4, 6]
];

let gameState = {
  board: Array(9).fill(''),
  currentPlayer: 'X',
  gameActive: true,
  mode: 'pvp',
  aiDifficulty: 'unbeatable',
  theme: 'standard',
  moveHistory: [],
  scores: { p1: 0, p2: 0, ties: 0 },
  isDarkMode: true,
  soundOn: true
};

try {
  const savedScores = localStorage.getItem('ttt_scores');
  if (savedScores) gameState.scores = JSON.parse(savedScores);
} catch (e) {}

const boardEl = document.getElementById('board');
const statusBanner = document.getElementById('status-banner');
const turnIconContainer = document.getElementById('turn-icon-container');
const turnText = document.getElementById('turn-text');
const turnSubtext = document.getElementById('turn-subtext');
const undoBtn = document.getElementById('undo-btn');
const restartBtn = document.getElementById('restart-btn');
const modePvpBtn = document.getElementById('mode-pvp');
const modeAiBtn = document.getElementById('mode-ai');
const aiDiffContainer = document.getElementById('ai-difficulty-container');
const aiDiffSelect = document.getElementById('ai-difficulty');
const markerThemeSelect = document.getElementById('marker-theme');
const winLineSvg = document.getElementById('winning-line-svg');
const winLine = document.getElementById('win-line');
const soundBtn = document.getElementById('sound-btn');
const soundIcon = document.getElementById('sound-icon');
const themeBtn = document.getElementById('theme-btn');
const themeIcon = document.getElementById('theme-icon');

const scoreP1El = document.getElementById('score-p1');
const scoreP2El = document.getElementById('score-p2');
const scoreTiesEl = document.getElementById('score-ties');
const scoreP1Label = document.getElementById('score-p1-label');
const scoreP2Label = document.getElementById('score-p2-label');
const resetScoreBtn = document.getElementById('reset-score-btn');

function createBoard() {
  boardEl.innerHTML = '';
  for (let i = 0; i < 9; i++) {
    const cell = document.createElement('button');
    cell.classList.add(
      'cell',
      'w-full', 'h-full', 'rounded-2xl', 'bg-slate-950/60', 'hover:bg-slate-800/80',
      'border', 'border-slate-800/80', 'flex', 'items-center', 'justify-center',
      'text-4xl', 'font-black', 'transition-all', 'duration-200', 'focus:outline-none',
      'focus:ring-2', 'focus:ring-indigo-500/50'
    );
    cell.dataset.index = i;
    cell.addEventListener('click', () => handleCellClick(i));
    boardEl.appendChild(cell);
  }
}

function updateUI() {
  const symbols = SYMBOL_SETS[gameState.theme];
  const cells = boardEl.querySelectorAll('.cell');

  gameState.board.forEach((val, idx) => {
    const cell = cells[idx];
    if (val === 'X') {
      cell.textContent = symbols.x;
      cell.className = cell.className.replace(/text-\S+/g, '');
      cell.classList.add('text-blue-400', 'animate-pop-in');
    } else if (val === 'O') {
      cell.textContent = symbols.o;
      cell.className = cell.className.replace(/text-\S+/g, '');
      cell.classList.add('text-pink-400', 'animate-pop-in');
    } else {
      cell.textContent = '';
    }
  });

  scoreP1Label.textContent = `Player 1 (${symbols.x})`;
  scoreP2Label.textContent = gameState.mode === 'ai' ? `AI (${symbols.o})` : `Player 2 (${symbols.o})`;
  scoreP1El.textContent = gameState.scores.p1;
  scoreP2El.textContent = gameState.scores.p2;
  scoreTiesEl.textContent = gameState.scores.ties;

  if (gameState.gameActive) {
    const isX = gameState.currentPlayer === 'X';
    const currentSymbol = isX ? symbols.x : symbols.o;
    turnIconContainer.textContent = currentSymbol;
    turnIconContainer.className = `text-2xl font-extrabold ${isX ? 'text-blue-400' : 'text-pink-400'} `;
    statusBanner.className = `w-full p-4 rounded-2xl glass-card flex items-center justify-between border-l-4 transition-all duration-300
      ${isX ? 'border-blue-500' : 'border-pink-500'} `;
  }

  if (gameState.mode === 'ai' && gameState.currentPlayer === 'O') {
    turnText.textContent = "AI is thinking...";
  }
}

```

```

        turnSubtext.textContent = "Calculating best move";
    } else {
        turnText.textContent = `${isX ? 'Player 1' : 'Player 2'}'s Turn`;
        turnSubtext.textContent = "Click any open spot";
    }
}
undoBtn.disabled = gameState.moveHistory.length === 0 || !gameState.gameActive;
}

function handleCellClick(index) {
    if (!gameState.gameActive || gameState.board[index] !== '') return;
    if (gameState.mode === 'ai' && gameState.currentPlayer === 'O') return;
    executeMove(index, gameState.currentPlayer);
    if (gameState.gameActive && gameState.mode === 'ai' && gameState.currentPlayer === 'O') {
        setTimeout(() => {
            makeAIMove();
        }, 350);
    }
}

function executeMove(index, player) {
    gameState.board[index] = player;
    gameState.moveHistory.push(index);
    audio.move();
    const winResult = checkWin(gameState.board);
    if (winResult) {
        endGame(winResult);
    } else if (isBoardFull(gameState.board)) {
        endGame({ winner: 'draw' });
    } else {
        gameState.currentPlayer = gameState.currentPlayer === 'X' ? 'O' : 'X';
        updateUI();
    }
}

function checkWin(board) {
    for (let combo of WINNING_COMBOS) {
        const [a, b, c] = combo;
        if (board[a] && board[a] === board[b] && board[a] === board[c]) {
            return { winner: board[a], combo };
        }
    }
    return null;
}

function isBoardFull(board) {
    return board.every(cell => cell !== '');
}

function endGame(result) {
    gameState.gameActive = false;
    if (result.winner === 'draw') {
        // Remaining game logic was not included in the supplied code.
    }
}

```